

```

// 伴窝 / Wayhouse
// 骨架版本 v0.3:
// 1. 魔法棒扩展菜单里加入口（跟玩伴小屋同位置）
// 2. 悬浮球入口：可拖拽、可上传 PNG 头像、圆形/方形可切换、大小可调
// 3. 小游戏加载器：iframe 沙盒方式，跟主运行时隔离，避免卡顿

import { builtInGames, gamesListHTML, loadGameIntoIframe } from
  './src/games.js';
import { normalizeBaseUrl, cleanText, fetchModelList, filterModels,
  createPreset } from './src/apiConfig.js';

const MODULE_NAME = 'wayhouse';
const EXT_VERSION = '0.4.0'; // 每次改动都会提一下，面板标题旁边能直接看到，方便确认
更新是否生效
const MENU_ID = 'wayhouse-menu-item';
const MENU_SELECTORS = [
  '#extensionsMenu',
  '#extensionMenuItems',
  '.extensions_block',
  '#extension_settings',
  '#extensionsMenuList',
];

const DEFAULT_SETTINGS = {
  floatingBallEnabled: true,
  floatingBallX: 18,
  floatingBallY: 160,
  floatingBallSize: 56,
  floatingBallShape: 'circle', // 'circle' | 'square'
  floatingBallImage: '', // base64 dataURL, 用户自己传的图
  customGames: [], // 用户自己添加的外链游戏 {name, icon, file, description}
  apiConfig: {
    mode: 'main', // 'main' 跟随酒馆主设置 | 'custom' 自定义接口
    presets: [], // [{id, name, baseUrl, apiKey, model}]
    activePresetId: null,
  },
};

function getContext() {
  return SillyTavern.getContext();
}

```

```

function getSettings() {
  const { extensionSettings } = getContext();
  if (!extensionSettings[MODULE_NAME]) {
    extensionSettings[MODULE_NAME] = structuredClone(DEFAULT_SETTINGS);
  }
  // 补齐新增字段，兼容老设置
  for (const key of Object.keys(DEFAULT_SETTINGS)) {
    if (extensionSettings[MODULE_NAME][key] === undefined) {
      extensionSettings[MODULE_NAME][key] =
structuredClone(DEFAULT_SETTINGS[key]);
    }
  }
  const apiCfg = extensionSettings[MODULE_NAME].apiConfig || {};
  if (apiCfg.mode === undefined) apiCfg.mode = 'main';
  if (!Array.isArray(apiCfg.presets)) apiCfg.presets = [];
  if (apiCfg.activePresetId === undefined) apiCfg.activePresetId = null;
  extensionSettings[MODULE_NAME].apiConfig = apiCfg;
  return extensionSettings[MODULE_NAME];
}

function saveSettings() {
  getContext().saveSettingsDebounce();
}

let panel = null;
let isVisible = false;
let floatBall = null;

// 弹窗单独挂在 document.body 下，不作为面板的子元素。
// 面板本身是 position:fixed，嵌套在里面的 fixed 弹窗在部分手机浏览器/WebView
// 上会被错误裁剪/错位（跟悬浮球同理，悬浮球从一开始就是挂在 body 下才没出过问题）。
function createModalsRoot() {
  if (document.querySelector('#wh-modals-root')) return;
  const root = document.createElement('div');
  root.id = 'wh-modals-root';
  root.innerHTML = `
    <div class="wh-modal-overlay" id="wh-game-modal-overlay"
style="display:none">
      <div class="wh-modal">
        <div class="wh-modal-title" id="wh-game-modal-title">添加游戏</div>

        <label class="wayhouse-row">
          <span>名称</span>

```

```

        <input type="text" id="wh-modal-name" maxlength="10"
placeholder="游戏名字">
    </label>

    <div class="wayhouse-row">
        <span>图标</span>
        <div class="wh-modal-icon-picker">
            <input type="text" id="wh-modal-emoji" maxlength="4"
placeholder="emoji">
            <label class="wayhouse-upload-btn wh-modal-icon-upload-btn">
                传图片
                <input type="file" id="wh-modal-icon-file" accept="image/*"
hidden>
            </label>
        </div>
    </div>
    <div class="wh-modal-icon-preview-row">
        <span>预览: </span>
        <span id="wh-modal-icon-preview">🎮</span>
        <button id="wh-modal-icon-clear" class="wh-modal-icon-clear"
style="display:none">清除图片</button>
    </div>

    <label class="wayhouse-row wh-modal-url-row">
        <span>游戏链接</span>
    </label>
    <input type="text" id="wh-modal-url" class="wh-modal-url-input"
placeholder="https://...">

    <div class="wh-modal-btns">
        <button id="wh-modal-delete" class="wh-modal-delete-btn"
style="display:none">删除</button>
        <button id="wh-modal-cancel" class="wh-modal-cancel-btn">取消
</button>
        <button id="wh-modal-save" class="wh-modal-save-btn">保存</button>
    </div>
</div>
</div>
`;
document.body.appendChild(root);
bindGameModalUI(document);
}

function createPanel() {

```

```

const cfg = getSettings();
panel = document.createElement('div');
panel.id = 'wayhouse-panel';
panel.innerHTML = `
  <div class="wayhouse-header">
    <span>伴窝 · Wayhouse <small class="wh-version-tag">v${EXT_VERSION}
</small></span>
    <button class="wayhouse-close" title="关闭">×</button>
  </div>
  <div class="wayhouse-tabs">
    <button class="wh-tab active" data-tab="home">主页</button>
    <button class="wh-tab" data-tab="games">小游戏</button>
    <button class="wh-tab" data-tab="api">接口</button>
  </div>
  <div class="wayhouse-body">
    <div class="wh-section" data-section="home">
      <p>骨架已跑通 

```

```

    </div>

    <div class="wayhouse-row">
        <span>自定义图片</span>
        <label class="wayhouse-upload-btn">
            选择 PNG
            <input type="file" id="wh-float-image"
accept="image/png,image/jpeg" hidden>
        </label>
    </div>

    <div class="wayhouse-row" ${cfg.floatingBallImage ? '' :
'style="display:none"} id="wh-float-clear-row">
        <span></span>
        <button class="wayhouse-clear-btn" id="wh-float-clear">清除图片
</button>
    </div>
</div>
</div>
</div>

    <div class="wh-section" data-section="games" style="display:none"
id="wh-games-section">
        <div class="wh-games-scroll" id="wh-games-scroll">
            <p class="wh-games-tip">提示: 右上角 × 关闭面板不会清空游戏进度, 点悬浮球
能随时回来接着玩; "返回列表"才会重新开始。长按卡片可编辑。</p>
            <div class="wh-games-grid" id="wh-games-
grid">${gamesListHTML(cfg.customGames)}</div>
            <button class="wayhouse-upload-btn wh-add-game-btn" id="wh-add-
game">+ 添加外链游戏</button>
        </div>

        <div class="wh-game-frame-wrap" id="wh-game-frame-wrap"
style="display:none">
            <div class="wh-gen-notify" id="wh-gen-notify"
style="display:none">
                <span>楼层生成完成啦, 要去看看吗? </span>
                <div class="wh-gen-notify-btns">
                    <button id="wh-gen-notify-view">去看酒馆</button>
                    <button id="wh-gen-notify-dismiss">继续玩</button>
                </div>
            </div>
            <div class="wh-game-frame-header">
                <button class="wh-game-back" id="wh-game-back">← 返回列表 (会清空当
前进度) </button>
            </div>
        </div>
    </div>

```

```

        <span class="wh-game-title" id="wh-game-title"></span>
    </div>
    <iframe
        class="wh-game-iframe"
        id="wh-game-iframe"
        frameborder="0"
        sandbox="allow-scripts allow-same-origin allow-forms allow-
pointer-lock allow-orientation-lock allow-popups allow-modals allow-
downloads"
        allow="accelerometer; gyroscope; gamepad; fullscreen; autoplay"
        loading="lazy"
        referrerpolicy="no-referrer-when-downgrade"></iframe>
    </div>
</div>

<div class="wh-section" data-section="api" style="display:none">
    <div class="wayhouse-row">
        <span>使用方式</span>
        <select id="wh-api-mode">
            <option value="main" ${cfg.apiConfig.mode === 'main' ?
'selected' : ''}>跟随酒馆主设置</option>
            <option value="custom" ${cfg.apiConfig.mode === 'custom' ?
'selected' : ''}>自定义接口</option>
        </select>
    </div>

    <div id="wh-api-custom-block" style="${cfg.apiConfig.mode ===
'custom' ? '' : 'display:none'}">
        <label class="wayhouse-row" style="flex-direction:column;align-
items:flex-start;gap:4px;">
            <span>接口地址 Base URL</span>
            <input type="text" id="wh-api-baseurl" class="wh-modal-url-
input" placeholder="https://xxx.com/v1">
        </label>
        <label class="wayhouse-row" style="flex-direction:column;align-
items:flex-start;gap:4px;">
            <span>API Key</span>
            <input type="password" id="wh-api-key" class="wh-modal-url-
input" placeholder="sk-...">
        </label>
        <label class="wayhouse-row" style="flex-direction:column;align-
items:flex-start;gap:4px;">
            <span>模型</span>
            <div style="display:flex;gap:6px;width:100%;">

```

```

        <input type="text" id="wh-api-model" class="wh-modal-url-
input" style="flex:1;margin-bottom:0;" placeholder="模型 ID">
        <button class="wayhouse-upload-btn" id="wh-api-fetch-models"
style="flex-shrink:0;">拉取模型</button>
    </div>
</label>
<div id="wh-api-status" class="wh-api-status"></div>

    <div id="wh-model-inline-wrap" style="display:none;margin-
bottom:10px;">
        <input type="text" id="wh-model-inline-filter" class="wh-modal-
url-input" placeholder="输入关键字筛选/锁定">
        <div id="wh-model-inline-list" class="wh-model-list"></div>
    </div>

    <button class="wayhouse-upload-btn" id="wh-api-save-preset"
style="width:100%;margin-top:10px;">+ 另存为新预设</button>

    <div class="wayhouse-settings-title" style="margin-top:16px;">已保
存预设</div>
    <div id="wh-api-preset-list" class="wh-api-preset-list"></div>
</div>
</div>
</div>
`;
panel.querySelector('.wayhouse-close').addEventListener('click',
hidePanel);
bindSettingsUI(panel);
bindTabsUI(panel);
bindGamesUI(panel);
bindApiSettingsUI(panel);
document.body.appendChild(panel);
createModalsRoot();
}

function bindTabsUI(root) {
    root.querySelectorAll('.wh-tab').forEach(btn => {
        btn.addEventListener('click', () => {
            root.querySelectorAll('.wh-tab').forEach(b =>
b.classList.remove('active'));
            btn.classList.add('active');
            const tab = btn.dataset.tab;

```

```

        root.querySelectorAll('.wh-section').forEach(sec => {
            sec.style.display = sec.dataset.section === tab ? 'flex' : 'none';
        });
    });
});
}

```

```

const LONG_PRESS_MS = 500;

```

```

function bindGamesUI(root) {
    const cfg = getSettings();
    const grid = root.querySelector('#wh-games-grid');
    const frameWrap = root.querySelector('#wh-game-frame-wrap');
    const iframe = root.querySelector('#wh-game-iframe');
    const titleEl = root.querySelector('#wh-game-title');

    let longPressTimer = null;
    let longPressFired = false;
    let pressStartX = 0;
    let pressStartY = 0;

    grid.addEventListener('pointerdown', e => {
        const item = e.target.closest('.wh-game-item');
        if (!item || item.dataset.customIndex === undefined) return; // 内置游戏不支持长按编辑
        longPressFired = false;
        pressStartX = e.clientX;
        pressStartY = e.clientY;
        longPressTimer = setTimeout(() => {
            longPressFired = true;
            openGameModal(document, 'edit', Number(item.dataset.customIndex));
        }, LONG_PRESS_MS);
    });

    const cancelLongPress = () => {
        if (longPressTimer) { clearTimeout(longPressTimer); longPressTimer = null; }
    };

    grid.addEventListener('pointermove', e => {
        if (!longPressTimer) return;
        if (Math.abs(e.clientX - pressStartX) + Math.abs(e.clientY - pressStartY) > 10) cancelLongPress();
    });
}

```

```

grid.addEventListener('pointerup', cancelLongPress);
grid.addEventListener('pointercancel', cancelLongPress);

grid.addEventListener('click', e => {
  if (longPressFired) { longPressFired = false; return; } // 长按触发过，这次
  点击不响应

  const delBtn = e.target.closest('.wh-game-del');
  if (delBtn) {
    const idx = Number(delBtn.dataset.customIndex);
    if (confirm('确定删除这个游戏? ')) {
      cfg.customGames.splice(idx, 1);
      saveSettings();
      grid.innerHTML = gamesListHTML(cfg.customGames);
    }
    return;
  }
  const item = e.target.closest('.wh-game-item');
  if (!item) return;
  const url = item.dataset.game;
  const name = item.dataset.name;
  grid.style.display = 'none';
  root.querySelector('#wh-games-scroll').style.display = 'none';
  frameWrap.style.display = 'flex';
  root.querySelector('#wh-games-section').style.overflow = 'hidden';
  titleEl.textContent = name;
  loadGameIntoIframe(iframe, url, name);
});

root.querySelector('#wh-game-back').addEventListener('click', () => {
  frameWrap.style.display = 'none';
  grid.style.display = 'grid';
  root.querySelector('#wh-games-scroll').style.display = '';
  root.querySelector('#wh-games-section').style.overflow = 'auto';
  iframe.srcdoc = '';
  hideGenNotify();
});

root.querySelector('#wh-add-game').addEventListener('click', () => {
  openGameModal(document, 'add', null);
});

root.querySelector('#wh-gen-notify-view').addEventListener('click', () =>
{

```

```

        hideGenNotify();
        hidePanel();
    });
    root.querySelector('#wh-gen-notify-dismiss').addEventListener('click',
hideGenNotify);
}

// ===== 独立 API 设置 =====
function escapeHtmlLocal(str) {
    return String(str ?? '').replace(/([<>"']/g, c => ({ '&': '&amp;', '<':
'&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));
}

let apiPresetDeleteHoldTimer = null;

function bindApiSettingsUI(root) {
    const cfg = getSettings();
    const modeSelect = root.querySelector('#wh-api-mode');
    const customBlock = root.querySelector('#wh-api-custom-block');
    const baseUrlInput = root.querySelector('#wh-api-baseurl');
    const keyInput = root.querySelector('#wh-api-key');
    const modelInput = root.querySelector('#wh-api-model');
    const statusEl = root.querySelector('#wh-api-status');
    const presetListEl = root.querySelector('#wh-api-preset-list');

    function fillFormFromActivePreset() {
        const preset = cfg.apiConfig.presets.find(p => p.id ===
cfg.apiConfig.activePresetId);
        baseUrlInput.value = preset?.baseUrl || '';
        keyInput.value = preset?.apiKey || '';
        modelInput.value = preset?.model || '';
    }
    fillFormFromActivePreset();

    function renderPresetList() {
        const presets = cfg.apiConfig.presets;
        if (!presets.length) {
            presetListEl.innerHTML = `<div class="wh-games-empty">还没有保存的预设
</div>`;
            return;
        }
        presetListEl.innerHTML = presets
            .map(p => `
                <div class="wh-preset-item">

```

```

        <span class="wh-preset-
name">${escapeHtmlLocal(p.name)}${cfg.apiConfig.activePresetId === p.id ? '
<b>· 使用中</b>' : ''}</span>
        <div class="wh-preset-actions">
            <button class="wh-preset-apply" data-id="${p.id}">使用</button>
            <button class="wh-preset-del" data-id="${p.id}">删除</button>
        </div>
    </div>`)
    .join('');
}
renderPresetList();

modeSelect.addEventListener('change', () => {
    cfg.apiConfig.mode = modeSelect.value;
    saveSettings();
    customBlock.style.display = cfg.apiConfig.mode === 'custom' ? '' :
'none';
});

root.querySelector('#wh-api-fetch-models').addEventListener('click', async
() => {
    const baseUrl = normalizeBaseUrl(baseUrlInput.value);
    const apiKey = cleanText(keyInput.value);
    if (!baseUrl) { statusEl.textContent = '请先填写接口地址'; return; }
    statusEl.textContent = '正在拉取模型列表...';
    const result = await fetchModelList(baseUrl, apiKey);
    if (!result.ok) { statusEl.textContent = result.error; return; }
    statusEl.textContent = `拉到 ${result.models.length} 个模型`;
    showInlineModelList(root, result.models, chosen => {
        modelInput.value = chosen;
        modelInput.dispatchEvent(new Event('change'));
        root.querySelector('#wh-model-inline-wrap').style.display = 'none';
    });
});

root.querySelector('#wh-api-save-preset').addEventListener('click', () =>
{
    const name = prompt('给这套配置起个名字（备注）：');
    if (!name) return;
    const preset = createPreset(name, {
        baseUrl: baseUrlInput.value,
        apiKey: keyInput.value,
        model: modelInput.value,
    });
});

```

```

    cfg.apiConfig.presets.push(preset);
    cfg.apiConfig.activePresetId = preset.id;
    saveSettings();
    renderPresetList();
  });

  presetListEl.addEventListener('click', e => {
    const applyBtn = e.target.closest('.wh-preset-apply');
    if (applyBtn) {
      cfg.apiConfig.activePresetId = applyBtn.dataset.id;
      saveSettings();
      fillFormFromActivePreset();
      renderPresetList();
      return;
    }
    const delBtn = e.target.closest('.wh-preset-del');
    if (delBtn) {
      if (!confirm('确定删除这个预设? ')) return;
      const id = delBtn.dataset.id;
      cfg.apiConfig.presets = cfg.apiConfig.presets.filter(p => p.id !==
id);
      if (cfg.apiConfig.activePresetId === id) cfg.apiConfig.activePresetId
= null;
      saveSettings();
      renderPresetList();
    }
  });

  // 输入框改动即时存草稿到当前激活预设（如果有），避免切走 Tab 丢内容
  [baseUrlInput, keyInput, modelInput].forEach(input => {
    input.addEventListener('change', () => {
      const preset = cfg.apiConfig.presets.find(p => p.id ===
cfg.apiConfig.activePresetId);
      if (!preset) return;
      preset.baseUrl = normalizeBaseUrl(baseUrlInput.value);
      preset.apiKey = cleanText(keyInput.value);
      preset.model = cleanText(modelInput.value);
      saveSettings();
    });
  });
}

function showInlineModelList(root, models, onPick) {
  const wrap = root.querySelector('#wh-model-inline-wrap');

```

```

const listEl = root.querySelector('#wh-model-inline-list');
const filterInput = root.querySelector('#wh-model-inline-filter');
filterInput.value = '';

function render() {
  const filtered = filterModels(models, filterInput.value);
  listEl.innerHTML = filtered.length
    ? filtered.map(m => `
```

```

const game = cfg.customGames[index];
title.textContent = '编辑游戏';
nameInput.value = game.name || '';
urlInput.value = game.file || '';
if (game.icon && game.icon.startsWith('data:image')) {
  modalIconDataURL = game.icon;
  emojiInput.value = '';
  preview.innerHTML = `

```

```

fileInput.addEventListener('change', e => {
  const file = e.target.files && e.target.files[0];
  if (!file) return;
  resizeImageToDataURL(file, 128).then(dataUrl => {
    modalIconDataURL = dataUrl;
    preview.innerHTML = ``;
    clearBtn.style.display = '';
  });
});

clearBtn.addEventListener('click', () => {
  modalIconDataURL = '';
  fileInput.value = '';
  preview.textContent = emojiInput.value || '🎮';
  clearBtn.style.display = 'none';
});

root.querySelector('#wh-modal-cancel').addEventListener('click', () => {
  closeGameModal(root);
  root.querySelector('#wh-game-modal-overlay').addEventListener('click', e
=> {
    if (e.target.id === 'wh-game-modal-overlay') closeGameModal(root);
  });

root.querySelector('#wh-modal-save').addEventListener('click', () => {
  const cfg = getSettings();
  const name = root.querySelector('#wh-modal-name').value.trim();
  const url = root.querySelector('#wh-modal-url').value.trim();
  const emoji = root.querySelector('#wh-modal-emoji').value.trim();
  if (!name || !url) {
    alert('名称和链接不能为空');
    return;
  }
  const icon = modalIconDataURL || emoji || '🎮';
  const entry = { name, icon, file: url, description: name };

  if (modalMode === 'edit') {
    cfg.customGames[modalEditIndex] = entry;
  } else {
    cfg.customGames.push(entry);
  }
  saveSettings();
  root.querySelector('#wh-games-grid').innerHTML =

```

```

gamesListHTML(cfg.customGames);
    closeGameModal(root);
  });

  root.querySelector('#wh-modal-delete').addEventListener('click', () => {
    if (!confirm('确定删除这个游戏? ')) return;
    const cfg = getSettings();
    cfg.customGames.splice(modalEditIndex, 1);
    saveSettings();
    root.querySelector('#wh-games-grid').innerHTML =
gamesListHTML(cfg.customGames);
    closeGameModal(root);
  });
}

function showGenNotify() {
  if (!panel) return;
  const bar = panel.querySelector('#wh-gen-notify');
  if (bar) bar.style.display = 'flex';
}

function hideGenNotify() {
  if (!panel) return;
  const bar = panel.querySelector('#wh-gen-notify');
  if (bar) bar.style.display = 'none';
}

// ===== 楼层生成完成提醒 =====
let genNotifyBound = false;
let lastNotifySignature = null;

function bindGenerationNotify() {
  if (genNotifyBound) return;
  const context = getContext();
  const eventTypes = context.event_types || {};
  const eventName = eventTypes.MESSAGE_RECEIVED || 'MESSAGE_RECEIVED';
  if (!context.eventSource || typeof context.eventSource.on !== 'function')
return;
  context.eventSource.on(eventName, handleMessageReceived);
  genNotifyBound = true;
}

function handleMessageReceived(messageId) {
  try {

```

```
    if (!isVisible || !panel) return; // 面板没开着不用提醒
    const frameWrap = panel.querySelector('#wh-game-frame-wrap');
    if (!frameWrap || frameWrap.style.display === 'none') return; // 只在正玩游戏时提醒
```

```
    const context = getContext();
    const chat = context.chat;
    const msg = (chat && messageId != null && chat[messageId]) || (chat && chat.length ? chat[chat.length - 1] : null);
    if (!msg || msg.is_user) return; // 只关心角色的回复，不关心玩家自己发的消息

    const signature = String(messageId ?? 'latest') + '::' + String(msg.message || msg.message || '').slice(0, 60);
    if (signature === lastNotifySignature) return;
    lastNotifySignature = signature;

    showGenNotify();
  } catch (e) {
    console.warn('[伴窝] 生成完成提醒出错:', e);
  }
}
```

```
function bindSettingsUI(root) {
  const cfg = getSettings();

  root.querySelector('#wh-float-enabled').addEventListener('change', e => {
    cfg.floatingBallEnabled = e.target.checked;
    saveSettings();
    syncFloatBall();
  });

  root.querySelector('#wh-float-shape').addEventListener('change', e => {
    cfg.floatingBallShape = e.target.value;
    saveSettings();
    syncFloatBall();
  });

  const sizeInput = root.querySelector('#wh-float-size');
  const sizeLabel = root.querySelector('#wh-float-size-val');
  sizeInput.addEventListener('input', e => {
    const size = Number(e.target.value);
    cfg.floatingBallSize = size;
    sizeLabel.textContent = size + 'px';
    saveSettings();
  });
}
```

```

    syncFloatBall();
  });

  root.querySelector('#wh-float-image').addEventListener('change', e => {
    const file = e.target.files && e.target.files[0];
    if (!file) return;
    resizeImageToDataURL(file, 256).then(dataUrl => {
      cfg.floatingBallImage = dataUrl;
      saveSettings();
      syncFloatBall();
      const clearRow = root.querySelector('#wh-float-clear-row');
      if (clearRow) clearRow.style.display = '';
    });
  });

  root.querySelector('#wh-float-clear').addEventListener('click', () => {
    cfg.floatingBallImage = '';
    saveSettings();
    syncFloatBall();
    root.querySelector('#wh-float-clear-row').style.display = 'none';
    root.querySelector('#wh-float-image').value = '';
  });
}

// 把用户传的图压缩到指定最大边长, 避免存进设置里的 base64 太大
function resizeImageToDataURL(file, maxSize) {
  return new Promise((resolve, reject) => {
    const img = new Image();
    const reader = new FileReader();
    reader.onload = () => {
      img.onload = () => {
        const scale = Math.min(1, maxSize / Math.max(img.width,
img.height));
        const w = Math.max(1, Math.round(img.width * scale));
        const h = Math.max(1, Math.round(img.height * scale));
        const canvas = document.createElement('canvas');
        canvas.width = w;
        canvas.height = h;
        const ctx = canvas.getContext('2d');
        ctx.drawImage(img, 0, 0, w, h);
        resolve(canvas.toDataURL('image/png'));
      };
      img.onerror = reject;
      img.src = reader.result;
    };
  });
}

```

```

    };
    reader.onerror = reject;
    reader.readAsDataURL(file);
  });
}

function showPanel() {
  if (!panel) createPanel();
  panel.style.display = 'flex';
  isVisible = true;
}

function hidePanel() {
  if (panel) panel.style.display = 'none';
  isVisible = false;
}

function togglePanel() {
  isVisible ? hidePanel() : showPanel();
}

// ===== 魔法棒扩展菜单项（跟玩伴小屋同位置） =====
let menuRetries = 0;
let menuRetryTimer = null;

function addMenuItem() {
  if (menuRetryTimer) { clearTimeout(menuRetryTimer); menuRetryTimer = null; }
  menuRetries = 0;
  tryInjectMenu();
}

function tryInjectMenu() {
  if (document.querySelector('#' + MENU_ID)) {
    installMenuObserver();
    return;
  }
  let menu = null;
  for (const sel of MENU_SELECTORS) {
    const found = document.querySelector(sel);
    if (found) { menu = found; break; }
  }
  if (menu) {
    appendMenuItem(menu);
  }
}

```

```

    installMenuObserver();
    return;
}
menuRetries++;
if (menuRetries < 30) {
    const delay = menuRetries < 5 ? 1000 : menuRetries < 15 ? 2000 : 3000;
    menuRetryTimer = setTimeout(tryInjectMenu, delay);
} else {
    console.warn('[伴窝] 未找到扩展菜单容器, 停止注入。');
}
}

function appendMenuItem(menu) {
    if (document.querySelector('#' + MENU_ID)) return;
    const wrap = document.createElement('div');
    wrap.className = 'extension_container interactable';
    wrap.tabIndex = 0;
    wrap.innerHTML = `
        <div class="list-group-item flex-container flexGap5 interactable"
id="${MENU_ID}" title="伴窝">
            <div class="fa-fw fa-solid fa-house extensionsMenuExtensionButton">
</div>
            <span>伴窝</span>
        </div>
    `;
    wrap.querySelector('#' + MENU_ID).addEventListener('click', togglePanel);
    menu.appendChild(wrap);
}

let menuObserver = null;
function installMenuObserver() {
    if (menuObserver) return;
    menuObserver = new MutationObserver(() => {
        if (document.querySelector('#' + MENU_ID)) return;
        for (const sel of MENU_SELECTORS) {
            const found = document.querySelector(sel);
            if (found) { appendMenuItem(found); break; }
        }
    });
    menuObserver.observe(document.body, { childList: true, subtree: true });
}

// ===== 悬浮球 =====
function clampFloatPosition(x, y, size) {

```

```

const vw = window.innerWidth;
const vh = window.innerHeight;
const margin = 8;
const nx = Number.isFinite(x) ? x : DEFAULT_SETTINGS.floatingBallX;
const ny = Number.isFinite(y) ? y : DEFAULT_SETTINGS.floatingBallY;
return {
  x: Math.max(margin, Math.min(nx, Math.max(margin, vw - size - margin))),
  y: Math.max(margin, Math.min(ny, Math.max(margin, vh - size - margin))),
};
}

function createFloatBall() {
  floatBall = document.createElement('div');
  floatBall.id = 'wayhouse-float-ball';
  document.body.appendChild(floatBall);
  bindFloatBallDrag(floatBall);
}

function renderFloatBall() {
  if (!floatBall) return;
  const cfg = getSettings();
  const size = cfg.floatingBallSize;
  Object.assign(floatBall.style, {
    width: size + 'px',
    height: size + 'px',
    borderRadius: cfg.floatingBallShape === 'circle' ? '50%' : '14px',
  });
  floatBall.innerHTML = cfg.floatingBallImage
    ? ``
    : `<span class="wayhouse-float-fallback">🏠</span>`;
  const pos = clampFloatPosition(cfg.floatingBallX, cfg.floatingBallY,
size);
  floatBall.style.left = pos.x + 'px';
  floatBall.style.top = pos.y + 'px';
}

function syncFloatBall() {
  const cfg = getSettings();
  if (!cfg.floatingBallEnabled) {
    if (floatBall) floatBall.style.display = 'none';
    return;
  }
  if (!floatBall) createFloatBall();
  floatBall.style.display = 'flex';
}

```

```

    renderFloatBall();
}

function bindFloatBallDrag(btn) {
    let dragging = false;
    let moved = false;
    let startX = 0, startY = 0, originX = 0, originY = 0;

    btn.addEventListener('pointerdown', e => {
        if (e.button !== null && e.button !== 0) return;
        const rect = btn.getBoundingClientRect();
        dragging = true;
        moved = false;
        startX = e.clientX;
        startY = e.clientY;
        originX = rect.left;
        originY = rect.top;
        btn.classList.add('dragging');
        try { btn.setPointerCapture(e.pointerId); } catch (err) {}
        e.preventDefault();
    });

    btn.addEventListener('pointermove', e => {
        if (!dragging) return;
        const dx = e.clientX - startX;
        const dy = e.clientY - startY;
        if (Math.abs(dx) + Math.abs(dy) > 4) moved = true;
        const cfg = getSettings();
        const pos = clampFloatPosition(originX + dx, originY + dy,
cfg.floatingBallSize);
        btn.style.left = pos.x + 'px';
        btn.style.top = pos.y + 'px';
        e.preventDefault();
    });

    const finish = e => {
        if (!dragging) return;
        dragging = false;
        btn.classList.remove('dragging');
        try { btn.releasePointerCapture(e.pointerId); } catch (err) {}
        const cfg = getSettings();
        const rect = btn.getBoundingClientRect();
        const pos = clampFloatPosition(rect.left, rect.top,
cfg.floatingBallSize);

```

```

    cfg.floatingBallX = pos.x;
    cfg.floatingBallY = pos.y;
    saveSettings();
    if (!moved) togglePanel();
};
btn.addEventListener('pointerup', finish);
btn.addEventListener('pointercancel', finish);

window.addEventListener('resize', () => renderFloatBall());
}

function init() {
    addMenuItem();
    syncFloatBall();
    bindGenerationNotify();
    console.info('[伴窝] 扩展已加载:', MODULE_NAME);
}

function start() {
    if (typeof SillyTavern === 'undefined') {
        setTimeout(start, 500);
        return;
    }
    const context = SillyTavern.getContext();
    if (context?.eventSource?.on) {
        context.eventSource.on(context.event_types.APP_READY, init);
    } else {
        setTimeout(init, 1000);
    }
}

start();

```